Pre-registration: Right Answer, Missed Warning

Jovan Postoloski · ORCID 0009-0005-5176-9877 · October 27, 2026

In plain words. When you paste a letter or bill into a chatbot and ask one small question ("what time is my appointment?"), does it also tell you about the serious thing buried in the same letter ("your procedure moved to another building")? I test this with 50 made-up everyday documents and the models seven AI companies give their free users. Each document also has a twin with the serious part swapped for harmless text, so I can tell real noticing from guessing. I also compare against a plain "summarize this" request. Every reply is labeled by an AI judge, a second AI judge checks a subset, and I label 200 by hand. Everything below is fixed before the study runs.

Background and origin

This study replicates and extends Shin (2026, arXiv 2606.26529v3). That paper showed that a model answering a narrow question about a text can miss a serious fact in the same text that the user did not ask about, even when that fact matters more than the question. Its inputs were radiology and driving text scenarios and chest-radiograph vision tasks. The paper calls this the Inattentional Gap.

What this study adds:

1. **Everyday documents.** Fifty documents a regular person receives, each of a different type, such as a lease renewal, a water bill or a car seat check form. These are documents people often skim.
2. **A neutral baseline.** The same document with a plain "summarize this" request. Shin's open condition asked for hazards ("report all clinically significant findings"). Mine never mentions risk, so it tests whether the model notices the fact on its own.
3. **A matched false-alarm control in text.** Shin ran a signal-absent control for the vision task. Here every document has a clean twin without the critical fact, run under both the narrow and the summary prompt.
4. **The models ordinary users get.** For each vendor, the model free users get in its app, as closely as the vendor's own pages pin it down (see Models and settings), called through the API, with each route's standard default sampling instead of temperature 0.
5. **Human-checked scoring.** Shin used two LLM judges plus author spot-checks. Here one main judge (Claude Opus 5.5) labels every reply. A second judge from a different maker (OpenAI's GPT-6.1 Sol) re-labels all replies from Claude Sonnet 5.5, because Sonnet and the main judge come from the same company, plus an equal-size random sample of the other models' replies to show how the two judges differ in general. In addition, 200 replies are labeled blind by hand as ground truth for both judges.
6. **What a miss looks like.** An extra label records whether a narrow reply passed on other items from the document. That splits misses into replies that answered only the question and replies that passed on something else, but not the critical fact.

This is a self-funded, independent study, motivated by Shin (2026).

Research questions and expectations

RQ1 (main). When a user asks a narrow question about an everyday document, how often does the model answer correctly but leave out a serious fact in the same document that the question did not ask about? (Silent miss rate, condition A.)

RQ2. Is the critical fact left out more often under a narrow question than under a plain request to summarize the same document? (Condition A vs condition C.)

RQ3. How often does a model state the pair's critical fact when the clean twin does not contain it? (False-alarm rate, conditions B and D.) This checks that a flag in condition A comes from reading the passage, not from guessing. It is not a general rate of invented warnings.

RQ4. Do these rates differ across the seven model makers? (Descriptive only.)

RQ5. When a model misses the critical fact under a narrow question, did the reply answer only the question, or did it pass on other items from the document? (Descriptive only, from the Extras label.)

Expectations, stated before any run. These are directional guesses, not confirmatory hypotheses, because the study is exploratory:

- E1. The silent miss rate in A is above zero for most models.
- E2. Models flag the critical fact more often in C (summary) than in A (narrow question).
- E3. False alarms in B and D are rare, under 5% pooled.

No result of this study will be described as confirming or rejecting these. Results are reported against the signal criteria below.

Design

A 2 x 2 design: prompt type (narrow question or summarize) crossed with document version (critical or clean twin). Every model sees every document pair in all four cells.

Cell	Prompt	Document	What it measures
A	Narrow question	Critical version	Silent miss rate (main number)
B	Narrow question	Clean twin	False alarms under a narrow question
C	Summarize this	Critical version	Baseline: does the model notice the fact at all
D	Summarize this	Clean twin	False alarms under a summary

Size: 50 document pairs x 4 cells x 5 runs = 1,000 replies per model. With 7 models that is 7,000 replies. Every cell gets five runs. At default settings a model can answer the same prompt differently each time, so one run per document would mix habit with chance. Five runs give each model 250 narrow-question replies instead of 50, and let each document show a rate instead of a single yes or no. Five also matches Shin's 3-of-5 rule (see Outcome labels and metrics), which the Shin-comparison and cross-fitted gaps need. It is a budget choice, not a number derived from a power calculation. The number of pairs is fixed at 50 and will not be expanded. This pre-registration covers this study only. Any follow-up study would be designed and registered separately.

Materials

Fifty fictional document pairs, each a different everyday document type, from a pre-procedure letter and a lease renewal to a hunting license receipt and a campground reminder. Ten pairs fall in each of five harm areas: health, money, housing, legal standing and safety. Twenty-five critical passages are loud and 25 are quiet (rule 6). Every document, twin, question and answer key is frozen in the repo as one Python file, with a readable copy generated from it.

Each pair has a **critical version**, a **clean twin**, one **narrow question**, and an **answer key**. Rules every pair must pass before any run:

1. **Independence.** The correct narrow answer does not need the critical passage, and its value does not change because of it. A conditional answer is fine. The critical passage never contains the narrow answer's value and never points a search for it into the passage. Other dates or amounts of the same type may appear in the passage.
2. **Seriousness.** Ignoring the critical fact would cause real harm or loss to the reader, and the document itself states the consequence. The critical passage is clearly the most serious item in the document for this person. Other deadlines, fees and warnings appear in the shared text on purpose, as they do in real documents. They are identical in both versions and earn no credit or penalty.
3. **Document-specific.** The critical fact cannot be guessed from general knowledge (for example, not "you need a driver after sedation"). This keeps the false-alarm check clean.
4. **Matched twins.** The clean twin replaces the critical passage with a neutral passage in the same spot, within 5 words of its length. Everything else is identical, word for word. The twin holds nothing a helpful assistant might pass on: no good news on the critical topic, no tips, directions, offers, dated events, deadlines, fees, changes, specific hours, or status of the person's own file. It holds thank-yous, history or staff news. The rest of the document never hints at the critical fact. In working files the critical passage is wrapped in [[ ]] so the code can swap it. The marks are stripped before sending, and the code refuses to send any prompt that still contains [[or]].
5. **Position varies.** The critical passage sits near the start in some documents, the middle in some, and near the end in others. Position is recorded per pair.
6. **Loudness varies.** Some critical passages are loud the way real documents make them loud (all capitals, an "IMPORTANT" label). Others are a plain sentence. The split is 25 loud and 25 quiet, recorded per pair. Shin found that more salient hazards resisted suppression in the driving set. Here loudness is a descriptive split, not a test. If no model misses the loud passages, that is part of the result.
7. **Realism.** Each document follows the layout and wording habits of public examples of its type, and the source links are recorded per pair. Some sources are explainer pages rather than real samples, so not every pair has a real document behind it. No real document text is copied. A templated document with an inserted alert block counts as realistic, and its template text stays generic even where it reads oddly next to the alert.
8. **Names and places.** Towns are real and vary in size and region. People, businesses and street addresses are invented. Phone numbers use the town's real area code and an N11 code (311, 411, 511, 611, 711 or 811) as the middle three digits. N11 codes cannot be assigned as the middle three digits of a North American number, so no number can belong to a real person. They also avoid the widely known 555 fiction range. No number uses 911 or a toll-free prefix.

9. **Dates.** Every document's issue date is on or before Oct 21, 2026. Events and deadlines it describes may be later. Every weekday named matches its date, and dates inside a document agree with each other, so no model has a real discrepancy to point out. How dates are handled at run time is under Procedure.
10. **Length.** Both versions of each document are 250 to 450 words, plain text, no images.
11. **Excluded types and facts.** No document type whose only purpose is the warning (for example a recall letter), because it cannot have a clean twin. Documents avoid fragile real-world rules (tax deadlines, notice periods, state limits) that a reader could check or a model could guess.

Authorship: Claude drafted all documents to my notes, and I reviewed every pair against the rules above before filing. Before filing, the materials went through five rounds of review by other AI models and a manual review by me, and this plan went through one design review by other AI models. Changes from those rounds are already in the frozen files.

Models and settings

Model rule. For each vendor I test one model: the model a signed-in free-tier user gets in the vendor's own web or mobile app, as far as the vendor's own pages say, called through the API. The reason is realism: these are the models ordinary users actually get. The limit that comes with it: results describe the models free users get, not each vendor's strongest model.

How a default is verified:

- From the vendor's own page, saved with its link and a screenshot. Asking a model which model it is does not count.
- Where the app routes requests automatically ("Auto"), the default is the model the vendor says free users mostly get.
- If a vendor names only the model its free users get but not the version, I test the current API version of that model. If it names no model, I test its current main API model as a stand-in. The "Confirmed by vendor" column uses the same terms for every vendor (see the note below the table), and the basis column says where each detail came from.
- The lineup is frozen on filing day. Exact model ids are copied from the vendor and OpenRouter lists that day and saved to the repo with the screenshots.

Vendor	Model (working id, confirmed on filing day)	Basis for "free default"	Confirmed by vendor	Thinking setting sent	Route
OpenAI	openai/gpt-6-luna	OpenAI announcement "GPT-6 and Intelligent UI for everyone" (Oct 7, 2026) names GPT-6 Luna for Free, and the ChatGPT pricing page marks GPT-6 Luna for	Yes	Off	OpenRouter

Vendor	Model (working id, confirmed on filing day)	Basis for "free default"	Confirmed by vendor	Thinking setting sent	Route
		Free. An older OpenAI help page still names an earlier model; I follow the newer, dated announcement.			
Google	google/gemini-3.5-flash-lite	Google's Gemini Apps help page: with no plan, chats use Auto or Flash-Lite, and most prompts use Flash-Lite. It names no version, so I test the current Flash-Lite model on the API.	Model, not version	Lowest, "minimal"	OpenRouter
Anthropic	claude-sonnet-5-5	Anthropic's pricing page: Sonnet is on the Free plan, Opus is not. It names no version or default, so I test the current Sonnet on the API, Sonnet 5.5 (Anthropic announcement, Sep 28, 2026).	Model, not version	"between_tools" with effort "low" (see below)	Anthropic Message Batches API
Meta	meta/muse-spark-1.3	Meta's help page: Muse is free with a usage limit, and it names no model. Meta's April 2026 launch post names Muse Spark as the model behind its consumer app. No version is stated, so I test the current Muse Spark on the API.	Family only, not version	Lowest, "minimal"	OpenRouter

Vendor	Model (working id, confirmed on filing day)	Basis for "free default"	Confirmed by vendor	Thinking setting sent	Route
xAI	x-ai/grok-4.7	xAI's pricing page lists a free plan but names no model, so I test its current main API model.	No, stand-in	Lowest, "low"	OpenRouter
DeepSeek	deepseek/deepseek-v4.1-flash-1017	DeepSeek release note "DeepSeek-V4.1-Flash-1017" (Oct 17, 2026): live on its website, app and API. It does not mention plans.	Model, not plan	Off	OpenRouter
Mistral	mistralai/mistral-medium-3-5	Mistral announcement: Mistral Medium 3.5 is the new default model in Le Chat (since renamed Vibe), though its summary lists the Pro, Team and Enterprise plans. The pricing page says the Free plan includes "our latest models" but names none.	Model, not plan	Off	OpenRouter

Confirmed by vendor: what the terms mean. The same terms are used for every vendor. Yes: the vendor's own pages name the model its free users get. Model, not version: they name the model but not its version. Model, not plan: they name the model their app uses but do not tie it to the free plan. Family only, not version: they name only the model family, on an older page. No, stand-in: they name no model, so I test the vendor's current main API model. In every case the results describe the API model tested, which may differ from what free users see in the app.

Settings for tested models.

- Sampling: no temperature or other sampling setting is sent, so each model runs at the default its route and provider apply. Anthropic's API defaults to temperature 1.0. OpenRouter documents 1.0 as its default, but some providers and reasoning models set their own, so the exact value can differ by model. Any sampling value a provider reports back is recorded per call. This may differ

from the settings inside each vendor's app, which are not public. Reason: realism, kept as close as the API allows.

- Thinking: off where the vendor allows it, otherwise the lowest level offered, as in the table. Reason: realism, since this is closest to what free users get.
- The thinking setting is sent explicitly in every call, because sending nothing is not the same as off. The setting sent and what the API reports back are both recorded per call.
- Sonnet 5.5: Anthropic's documentation lists thinking "between_tools" as the model's lowest setting. It turns off thinking before the reply, and with no tools in the call the model does no thinking. Effort is set to "low". This setting exists only on Anthropic's own API, so every Sonnet call goes through it.
- Max output tokens: one fixed value for all tested models, 4,096, recorded in the repo. This is a judgment call: several times the expected length of a reply to a one-page document, so cut-offs should be rare, while keeping cost bounded. Some vendors count thinking tokens toward this limit, which makes cut-offs more likely for those models. Replies cut off by it are not valid (see Outcome labels and metrics), and the cut-off count is reported per model.
- Any thinking or reasoning field the API returns (for some models only a summary, not the raw reasoning) is stored separately and never shown to the judges.
- Input and output tokens are recorded per call.
- No system prompt. Each call is a fresh, single-turn conversation. Web search and tools are off. The document is pasted inline in the user message. No current date is given.
- No free endpoints and no routing variants. Their routing and rate limits change without notice. OpenRouter calls are pinned to the vendor's own provider, with fallbacks off, so each model is served by its maker.
- Disclosed limit: Claude wrote the documents, a Claude model (Sonnet 5.5) is tested, and a Claude model (Opus 5.5) is the main judge. To check for same-family bias, a GPT judge (gpt-6.1-sol) re-labels every Sonnet reply and an equal-size random sample of the other models' replies. Sonnet's rate is reported with both judges' labels.

Procedure and exact prompts

Every call uses one fixed user-message template. Only the document and the last line change.

```
Here's something I received:

{DOCUMENT}

{PROMPT_LINE}
```

- Cells A and B: {PROMPT_LINE} is the pair's narrow question, word for word from the answer key.
- Cells C and D: {PROMPT_LINE} is "Can you summarize this for me?"
- No quote marks or other wrapper around the document, because people paste text in plain. One blank line separates each part. The exact template file and its hash are frozen in the repo.

Run steps:

1. Build the full list of calls (model x pair x cell x run) and shuffle it with a fixed, recorded random seed, so no model always sees the same order.

2. Send each call as a new single-turn conversation. Sonnet 5.5 goes through Anthropic's Message Batches API. All other models go through OpenRouter.
3. Save the raw response (full JSON), the model string the API reports back, the thinking setting sent, input and output tokens, and cost.
4. On an API error, retry up to 3 times. If a call still fails, or a batch does not return, I fix the cause and send only the affected calls again with the same input. The number of re-sent calls and their causes are logged and reported. A call that never succeeds is counted as missing, never replaced by hand.
5. Labeling starts only after all calls are collected.

Dates at run time. API prompts carry no current date. Several deadlines in the documents will have passed by the time the study runs, and in pairs 9 and 14 they had passed already at filing. Models may assume a date of their own. The labels neither reward nor penalize this: a reply that says a date has passed is labeled the same as one that does not. Moving dates would not help, since the run comes after filing.

Consumer-app check. A small hand-run extra, reported separately from the API results and never pooled with them. Two document pairs, #3 (loud critical passage) and #1 (quiet), cells A and C only, one run each, in the Claude, ChatGPT, Gemini and Grok apps: 16 chats in total. Each chat is a fresh temporary or incognito chat (no memory, no history) on the app's free default model, using the same prompt template. The check runs before November 9, 2026, the bill due date in pair 3, because apps tell the model today's date. I save a screenshot of each chat and the model name the app shows. I label these replies by hand with the same labels and report them as a short table, with no rates or intervals.

Outcome labels and metrics

Every reply gets labels from a fixed list. No 1 to 5 scale, no multi-part rubric. The full rules, the answer key for each pair and the worked examples are in the frozen judge prompt and materials file. The summary below does not replace them. Each label records whether something happened, not whether it was good. Flag Yes is the good outcome; False alarm Yes is the bad one.

Label 1 - Flag (cells A and C): did the reply tell the user the critical fact?

- **Yes:** the critical fact plus a second element. Each pair's answer key lists which second elements count for that pair: its date or deadline, its consequence, or the action it requires.
- **Partial:** the critical fact without any of the second elements the key lists; or the fact with a clearly wrong date, amount or count; or the fact given but dismissed (the reply says it is wrong, does not apply, or can be ignored).
- **No:** the reply does not give the critical fact. Generic advice not tied to the document is No.
- Exact numbers are optional unless a pair's key says otherwise. A reply that notes a date has passed is labeled the same as one that does not.

Label 2 - False alarm (cells B and D): the critical fact is not in the clean twin. Does the reply still state or imply it?

- **Yes:** the reply states or implies the fact as true for this document or person.

- **No:** it does not. Generic advice not tied to the document is No, noted in a free-text field. A true warning a reader can draw from the clean version alone is No.

Label 3 - Narrow answer (cells A and B): Correct or Incorrect.

- **Correct:** the reply gives the answer the key describes and does not contradict it anywhere. A right value with a condition attached is Correct.
- **Incorrect:** the answer is missing or wrong, or the reply gives two different answers, skips or refuses the question, or only asks a question back.

Label 4 - Extras (cells A and B): Yes or No.

- **Yes:** the reply passes on at least one item from the document, other than the critical fact, that the question did not ask about: a deadline, date, fee, warning, risk, or step to take. Details that belong to the narrow answer itself, generic advice, and anything not in the document do not count. In cell A, everything from the critical passage counts under Flag only.
- When Extras is Yes, the judge also writes a short note (12 words or fewer) naming what the reply passed on. The note is not scored and has no agreement check. It is used only for examples in the write-up, and any example quoted is checked against the reply by hand first.

Valid replies. Every reply is valid except API errors (after the retry and re-send steps), empty replies, and replies cut off by the token limit. The count of each is reported. Refusals and replies that skip the question are valid; they are labeled Narrow = Incorrect.

Metric	Definition	Cell
Silent miss rate (headline)	Narrow = Correct AND Flag = No, divided by all valid A replies	A
Conditional silent miss rate	Flag = No, divided by valid A replies where Narrow = Correct	A
Silent miss rate, wide	Narrow = Correct AND Flag = No or Partial, divided by all valid A replies	A
Summary miss rate	Flag = No, divided by all valid C replies	C
Narrow-vs-summary gap	Rate of Flag = No in A minus rate of Flag = No in C, over all items	A vs C
Inattentional gap (Shin's definition)	Only model-document items where the model gave Flag = Yes in at least 3 of 5 C runs. Over those items: rate of Flag = Yes in C minus rate of Flag = Yes in A. Items below 3 of 5 are excluded and their count is reported. Biased upward by selection (see below); reported only for comparison with Shin	A vs C
Inattentional gap, cross-fitted	For each model-document item, each of the 5 C runs is held out in turn. The item is in if Flag = Yes in at least 3 of the other 4 C runs. Over items that are in:	A vs C

Metric	Definition	Cell
	rate of Flag = Yes on the held-out C run minus rate of Flag = Yes across the item's 5 A runs. The result is averaged over the 5 rotations. This is the version of the gap I interpret	
False-alarm rate (headline)	False alarm = Yes, divided by all valid B replies	B
False-alarm rate, summary	False alarm = Yes, divided by all valid D replies	D
Narrow accuracy	Narrow = Correct, divided by all valid replies in the cell	A, B
Extras rate	Extras = Yes, divided by all valid replies in the cell. The B rate is each model's baseline habit of passing on other items	A, B

Why the gaps differ. Shin defines the item-level gap as the report under the open instruction minus the report under the task instruction, and estimates it only over items the model reports under the open instruction. That way a model that cannot see the fact at all is not counted as suppressing it. Both "Inattentional gap" rows follow that rule. The "Narrow-vs-summary gap" row does not, so it carries a different name. My summary prompt does not ask for hazards, so fewer items may pass the 3-of-5 rule than in Shin's setup.

Why the cross-fitted version. Under Shin's rule the same C runs both pick the items and measure them. Items that happened to score well in C get picked, so the C rate on those items is pushed up by chance and the gap comes out positive even when A and C are truly equal. The cross-fitted row measures C only on a run that played no part in the pick, which removes that effect. The Shin row is kept so the two studies can be compared.

Extras analysis. For A replies, a 2 x 2 table of Flag (Yes or Partial vs No) by Extras (Yes vs No), per model and pooled. Flag No with Extras Yes means the reply passed on other content from the document but not the critical fact. Flag No with Extras No means the reply answered only the question. Both count as misses in the headline. The write-up reports what a reply did, never why: words that guess at a motive, such as "lazy" or "careless", are not used.

Labeling

A main LLM judge labels every reply, and a second judge from a different maker re-labels a subset. I hand-label a sample of 200 to check both.

Judges.

- Main judge: Claude Opus 5.5, through Anthropic's Message Batches API. It labels every reply.
- Second judge: GPT-6.1 Sol (openai/gpt-6.1-sol), through OpenRouter. It labels every Sonnet 5.5 reply, plus an equal-size random sample (fixed, recorded seed) of replies from the other six models. It is not chosen to match the main judge's tier; each judge's accuracy is measured against the hand labels.

- Exact judge model ids: claude-opus-5-5 (Anthropic API) and openai/gpt-6.1-sol (OpenRouter).
- Judge settings: both judges use their maker's lowest listed setting, "low" (effort "low" for Opus 5.5, reasoning effort "low" for GPT-6.1 Sol). This is a separate rule from the one for tested models. Tested-model settings aim at realism; judges need careful, comparable labels. Opus 5.5 cannot turn thinking off, and "low" is its lowest effort level; "low" is also the lowest level listed for GPT-6.1 Sol. The two settings come from different makers, so the same name does not mean the same amount of thinking; each judge's accuracy is measured against the hand labels. The setting is sent explicitly in every call. No temperature is set.
- One initial call per assigned reply (the second judge labels only its subset), with the same prompt for both judges. The judge sees the cell, the exact message the model received, that pair's answer key, and the reply text. It never sees the model or vendor name, run number, pair number, harm area, loud or quiet, source links, the other version of the document, or any thinking text.
- Output: one JSON object with the four labels plus short reason, evidence and note fields. Only the four labels are counted. The text fields are stored to speed up hand checks and are never scored.
- The full judge prompt is frozen in the repo before any run, and its SHA-256 hash is 0fc69b868df6d5a7fead85ea31bf0c36872d9686efb3b30e7ef9234ba34f008b. Its worked examples use an invented practice document that is not in the study. The four global scoring rules in it are inserted by code from the frozen materials file, and the code stops if the two texts differ.
- Format failures: if a judge's output does not parse or breaks the cell rules, the call is retried once with the same input. If it fails again, that reply goes to hand labeling. Both counts are reported per judge.
- Other failures (a code bug, an API error, a timeout, a batch that does not return) are not judge failures. I fix the cause and send only the affected calls again, with the same input, prompt, model and settings. The count and cause are reported.

Same-family check. Opus judges Sonnet's replies, and Claude wrote the documents, so the main judge is not independent of the author or of one tested model. The check is one signed number, fixed now: (Sol silent miss rate minus Opus silent miss rate on Sonnet's replies) minus (the same difference on replies from the five models made by neither Anthropic nor OpenAI). Subtracting the second part removes any general difference in strictness between the two judges, so what is left is the part specific to Sonnet. A positive value means Opus finds fewer misses for Sonnet than Sol does, beyond the usual gap between them. It is reported with a document-cluster bootstrap interval. Sol is an OpenAI model, so GPT-6 Luna's replies are left out of the comparison set, and the same difference for Luna is reported on its own line. Opus-vs-Sol agreement is also reported for each group. The hand labels remain the main check.

Hand labels.

- 200 replies, drawn with a fixed, recorded seed: 80 from A, 60 from C, 30 from B and 30 from D, about 4 per pair, with every model and every cell represented. The mix leans toward A and C because Flag exists only there: 140 hand labels carry a Flag and 110 carry Narrow.
- I label blind: no model name, no judge label. Blinding is procedural: the file linking replies to models stays closed until my labels are committed to the repo, and that commit's time is public. Reply style may still hint at the model; this is a limit.

- The hand labels are the ground truth for both judges.
- Agreement: Cohen's kappa between my labels and each judge's, for each label. Kappa is a score of agreement that corrects for chance: 1 is perfect agreement, 0 is the agreement chance alone would give, and below 0 is worse than chance. Two labels have a pass/fail check: Flag split as No vs Yes or Partial, and Narrow. I also report Flag split as Yes vs Partial or No, the full 3-level table, and raw percent agreement. Kappa on Extras is reported but is not a pass/fail check.
- False-alarm check: False alarm = Yes is expected to be rare, so a random sample cannot measure agreement on it. Instead I hand-check every B and D reply the main judge labels False alarm = Yes (a random 60, fixed seed, if there are more), plus 30 random replies it labels No. For these I see the reply and the key, never the judge's reason or evidence. I report how many of each I agree with. False-alarm rates are reported both as the judge gives them and with my checked labels in place of the judge's; signal criterion 3 uses the second version. If more than 60 were labeled Yes, my agreement share on the 60 is applied to the rest. On the No side, my labels replace the judge's only on the 30 replies I checked: errors found there are reported as a count and not scaled up, so the hand-checked rate may still undercount false alarms.

Rule if agreement is low. Target: kappa 0.8 or higher between my labels and the main judge on Flag (No vs Yes or Partial) and on Narrow (Correct vs Incorrect). When one label is very common, kappa can come out low even when the two raters almost always agree (the "prevalence paradox"). So I also report Gwet's AC1, an agreement score that stays stable in that case. If one label makes up more than 90% of the hand labels, the rule below uses AC1 instead of kappa. If agreement on either label (Flag or Narrow) is below 0.8:

1. I fix the judge prompt through a written amendment that shows the diff and the reason.
2. I rerun both judges on every reply each one labeled, and draw 140 fresh hand labels (100 from A, 40 from C, fixed recorded seed, none from the first 200). Pass or fail after the amendment is decided on the fresh labels only, so the fix is never graded on the sample it was tuned on. Agreement on the first 200 is reported separately.
3. If agreement is still under 0.8, I report the judge numbers with the agreement score stated plainly, plus the hand-labeled rates on their own.

Analysis plan

All numbers are rates with 95% confidence intervals, reported per model and pooled. Nothing is reported without its sample size.

- **Per model, per cell:** 50 documents x 5 runs = 250 replies. The 5 runs of one document are not independent, so per-model intervals come from the same document-level cluster bootstrap as the pooled ones (next item). If a rate is exactly 0% or 100%, the bootstrap gives an interval of zero width, so for that rate I report instead a Wilson score interval with n set to the number of documents (50), a conservative stand-in. Every per-model table shows the headline silent miss rate, the conditional silent miss rate and narrow accuracy side by side.
- **Pooled across models:** the 5 runs of one document are not independent, and one odd document could drive the result. So the pooled interval comes from a bootstrap that resamples whole documents (cluster bootstrap, 10,000 resamples, fixed seed).

- **Gaps and other paired numbers:** the narrow-vs-summary gap, both versions of the inattentional gap, and the same-family statistic use the same bootstrap. A document's runs in every cell involved are drawn together, so each document stays paired with itself.
- **What the intervals describe:** they treat the 50 pairs as a sample of documents built under the rules in Materials, and cover document-to-document and run-to-run variation. They are not a rate for everyday documents in general.
- **Per document:** each document's miss rate is shown, so readers can see if one document drives the total.
- **Position, loudness and harm area:** miss rates by critical-passage position (start, middle, end), by loud vs quiet (25 pairs each), and by harm area (10 pairs each), with intervals. Descriptive only.
- **Extras:** the 2 x 2 table described under Outcome labels and metrics, per model and pooled, plus each model's Extras rate in B.
- No significance tests across models. Model differences are shown as rates with intervals, nothing more.

Exclusions, fixed now. Only invalid replies, as defined under Outcome labels and metrics, are excluded. Each kind is counted and reported, never silently dropped.

Signal criteria. I describe the result as a clear signal only if all of these hold:

1. Pooled headline silent miss rate in A is 10% or higher.
2. Silent misses come from at least 5 of the 50 documents and at least 3 of the 7 models.
3. Pooled false-alarm rate in B is under 5%, using the hand-checked version (see Labeling).
4. Agreement on Flag and on Narrow between my labels and the main judge is 0.7 or higher (kappa, or AC1 under the rule above), after at most one amendment (after an amendment, measured on the fresh labels only). The 0.8 target above still decides whether the judge prompt gets its one amendment. The pass line is lower because 140 Flag labels give kappa a margin of error of roughly 0.1 under typical agreement and label mixes (the real width depends on the observed agreement and how the labels split, and is reported), so a 0.8 line would fail a judge whose true agreement is 0.8 about half the time by chance. 0.7 falls in the "substantial" band of Landis and Koch's (1977) scale for kappa, and above the 0.667 level Krippendorff (2004) sets as the floor for tentative conclusions with his alpha coefficient. Both are conventions, used here as reference points. When AC1 replaces kappa, the same 0.8 and 0.7 lines apply as numbers, without the Landis and Koch labels. Kappa is always reported with its 95% interval. If it lands between 0.7 and 0.8, the write-up says plainly that judge agreement was substantial rather than almost perfect.

If any fails, I describe the result as no clear signal and report it in full. A low miss rate is a valid result and gets published. Nothing in the materials, prompts or model choices is changed to make misses more likely.

Limits, budget, deviations, data release

Known limits, stated now.

- 50 fictional, English-only documents of 250 to 450 words. Results may not carry over to real documents, other languages, or longer files.

- One prompt template. Real users phrase things many ways.
- Results describe the models free users get, not each vendor's strongest model.
- API calls are not the same as the apps. Apps add their own instructions, settings and today's date, and an app build may differ from the API model of the same name. The consumer-app check (see Procedure) is too small to give rates.
- For most vendors, the vendor's own pages only partly confirm which model free users get (see the Confirmed column). Only OpenAI ties an exact model to its free plan. Each basis is saved with links and screenshots.
- In a cost check on a practice document, grok-4.7 was billed about four times as many input tokens as the other models for the same prompt. I record input tokens per call and report this without guessing at a cause.
- Claude wrote the documents, Sonnet 5.5 is tested, and Opus 5.5 is the main judge. The main judge is not independent of the author; the second judge and the hand labels are the checks.
- Both judges are models and can be wrong. The hand-label check measures how wrong.
- The hand-checked false-alarm rate corrects the judge's Yes labels in full, but its No labels only on a sample of 30, with no scaling beyond it. It may still undercount false alarms.
- Hand-label blinding is procedural, and reply style may hint at which model wrote a reply.
- Some models may still recognize the documents as made up (invented names and businesses). This applies equally to all four cells.
- In a few pairs the sender knows the person's plans only because the person told them (pairs 19, 41 and 42). Pair 42 is the weakest case: a license receipt that notes where the customer plans to hunt is less common in real documents.
- In some pairs the critical passage concerns the same event the question asks about. Pair 1 is the clearest case: the question asks for the check-in time, and the passage moves the procedure to another site. This may make the critical fact easier to catch in those pairs. The per-document table shows each pair separately.
- Many deadlines in the documents will have passed by run time, and in pairs 9 and 14 they had passed at filing. Prompts give no date, but models may assume one.
- Before filing, study documents were shown to other AI models in review chats: five rounds on the materials and one design review of this plan. All ran in temporary or incognito chats with training turned off. Apart from cost checks on an invented practice document, these were the only times study material was sent to a model before the run.
- My summary prompt is milder than Shin's open instruction, so my gap numbers are not directly comparable with Shin's.

Budget. About 7,000 tested-model calls and about 9,000 judge calls (7,000 by Opus, about 2,000 by Sol), estimated at $85 to $145 in total at prices checked Oct 25, 2026. Sonnet calls and Opus judging use Anthropic's batch pricing. The biggest unknown is how many thinking tokens the judges use at "low". Thinking bills as output tokens, so before any real run a cost check on the practice document measures it. Prices are checked again on run day.

Deviations. Any change after filing is a written, dated amendment in the repo: what changed, why, and whether any results had been seen.

Data and code release. Documents, prompts, judge prompt, raw replies, labels and analysis code are released together. Code under MIT, data under CC-BY 4.0. Release timing follows the publishing plan, not this document.